ASSIGNMENT-13.2

Name:B.Varun

Batch:38

H.No:2303A52020

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

Sample Code:

```
length = 8
width = 4
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Code:

C: > Users > varun > Downloads > Untitled-2.py > calculate_perimeter

```python
"""
Refactored Rectangle Properties Calculator

This module demonstrates the elimination of repetitive logic by encapsulating
area and perimeter calculations into reusable functions.
"""


def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle
        width (float): The width of the rectangle

    Returns:
        float: The area of the rectangle (length * width)
    """
    return length * width


def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle
        width (float): The width of the rectangle

    Returns:
        float: The perimeter of the rectangle (2 * (length + width))
    """
    return 2 * (length + width)


def print_rectangle_properties(length, width):
```

```python
    Print the area and perimeter of a rectangle.

    This function encapsulates the calculation and printing logic
    reducing code repetition and improving maintainability.

    Args:
        length (float): The length of the rectangle
        width (float): The width of the rectangle

    Returns:
        None
    """
    area = calculate_area(length, width)
    perimeter = calculate_perimeter(length, width)
    print("Area:", area)
    print("Perimeter:", perimeter)


# Main program
if __name__ == "__main__":
    # Rectangle 1: length = 8, width = 4
    print_rectangle_properties(8, 4)

    # Rectangle 2: length = 12, width = 6
    print_rectangle_properties(12, 6)
```

Output:

```
^C
PS C:\Users\varun\OneDrive\Desktop\python> & "C:/Progr
e" c:/Users/varun/Downloads/Untitled-2.py
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
PS C:\Users\varun\OneDrive\Desktop\python>
```

Task Description -2 （Refactoring – Modularizing Inline Calculations)

• Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Sample Code:

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

Code:

```
C: > Users > varun > Downloads > Untitled-2.py > ...
def calculate_discount(amount, discount_rate):
    -----------
    amount : float
        The original amount before discount
    discount_rate : float
        The discount rate as a decimal (e.g., 0.10 for 10%)

    Returns:
    --------
    float
        The discount amount
    """
    return amount * discount_rate


def calculate_final_amount(amount, discount_rate):
    """
    Calculate the final amount after applying discount.

    Parameters:
    -----------
    amount : float
        The original amount before discount
    discount_rate : float
        The discount rate as a decimal (e.g., 0.10 for 10%)

    Returns:
    --------
    float
        The final amount after discount
    """
    discount = calculate_discount(amount, discount_rate)
    return amount - discount


# Test with first amount
amount = 1000
final_amount = calculate_final_amount(amount, 0.10)
print("Final Amount:", final_amount)

# Test with second amount
amount = 2000
final_amount = calculate_final_amount(amount, 0.10)
print("Final Amount:", final_amount)
```

OUTPUT:

```
PS C:\Users\varun\OneDrive\Desktop\python> & "C:/Pr
ed-2.py
Final Amount: 900.0
Final Amount: 1800.0
PS C:\Users\varun\OneDrive\Desktop\python>
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

• Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

• Use AI to recommend more efficient data structures or logic simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
exists = False
for student in students:
if name == student:
exists = True
if exists:
print(name, "found")
else:
print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

CODE:

C: > Users > varun > Downloads > Untitled-2.py > ...

```python
# ============================================================================
# TASK 2: Refactoring – Modularizing Inline Calculations
# ============================================================================

def apply_discount(amount, discount_rate=0.10):
    """
    Calculate the final amount after applying a discount.

    Args:
        amount (float): The original amount
        discount_rate (float): The discount rate as a decimal (default: 0.10 for 10%)

    Returns:
        float: The final amount after discount
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount


# Test with sample data
print("=" * 50)
print("TASK 2: Modularizing Inline Calculations")
print("=" * 50)

amount1 = 1000
final_amount1 = apply_discount(amount1)
print(f"Amount: {amount1}, Final Amount: {final_amount1}")

amount2 = 2000
final_amount2 = apply_discount(amount2)
print(f"Amount: {amount2}, Final Amount: {final_amount2}")

print("\n")


# ============================================================================
# TASK 3: Refactoring – Improving Loop and Conditional Performance
# ============================================================================

import time

students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
```

C: > Users > varun > Downloads > Untitled-2.py > ...

```python
print("=" * 50)
print("TASK 3: Improving Loop and Conditional Performance")
print("=" * 50)

# Original Implementation (Inefficient - Nested Loops)
print("\n1. ORIGINAL IMPLEMENTATION (Nested Loops):")
print("-" * 50)

start_time_original = time.perf_counter()

for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(f"{name} found")
    else:
        print(f"{name} not found")

end_time_original = time.perf_counter()
original_time = (end_time_original - start_time_original) * 1000000  # Convert to microseconds


# Optimized Implementation (Using Set for O(1) Lookup)
print("\n2. OPTIMIZED IMPLEMENTATION (Using Set):")
print("-" * 50)

def find_students_optimized(check_list, students):
    """
    Efficiently check if students exist in the list using a set.

    Args:
        check_list (list): List of names to check
        students (list): List of enrolled students

    Returns:
        None: Prints results
    """
    students_set = set(students)  # Convert to set for O(1) lookup

    for name in check_list:
        if name in students_set:
            print(f"{name} found")
```

```python
        else:
            print(f"{name} not found")


start_time_optimized = time.perf_counter()
find_students_optimized(check_list, students)
end_time_optimized = time.perf_counter()
optimized_time = (end_time_optimized - start_time_optimized) * 1000000  # Convert to microseconds


# Performance Comparison
print("\n" + "=" * 50)
print("PERFORMANCE COMPARISON")
print("=" * 50)
print(f"Original Implementation Time:  {original_time:.4f} microseconds")
print(f"Optimized Implementation Time: {optimized_time:.4f} microseconds")
improvement = ((original_time - optimized_time) / original_time) * 100 if original_time > 0 else 0
print(f"Performance Improvement:       {improvement:.2f}%")
print(f"\nTime Complexity:")
print(f"  Original: O(m * n)  where m = {len(check_list)}, n = {len(students)}")
print(f"  Optimized: O(m + n) = O({len(check_list)} + {len(students)}) = O({len(check_list) + len(students)})")
print("=" * 50)
```

OUPUT:

```
PS C:\Users\varun\OneDrive\Desktop\python> & "C:/Prog
ed-2.py
Bhavya found

==================================================
PERFORMANCE COMPARISON
==================================================
Original Implementation Time:  250.0000 microseconds
Optimized Implementation Time: 194.8000 microseconds
Performance Improvement:       22.08%

Time Complexity:
  Original: O(m * n)  where m = 3, n = 4
  Optimized: O(m + n) = O(3 + 4) = O(7)
```

Task Description -4 (Refactoring – Replacing Hardcoded Values

with Constants)

• Task: Utilize AI assistance to improve maintainability by

replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing

results.

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

CODE:

```
C: > Users > varun > Downloads > Untitled-2.py > ...
# Task 4: Refactoring - Replacing Hardcoded Values with Constants
"""
This script demonstrates replacing hardcoded numeric values with
descriptive named constants for improved maintainability.
"""

# ============ CONSTANTS SECTION ============
# Define all hardcoded values as named constants at the beginning
RATE = 2                        # Interest rate (%)
TIME = 5                        # Time period (years)
FORMULA_DIVISOR = 100           # Divisor for percentage calculation

# Principal amounts for different calculations
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000


def calculate_simple_interest(principal, rate, time):
    """
    Calculate simple interest using the formula: (P * R * T) / 100

    Args:
        principal (float): The principal amount
        rate (float): The rate of interest per annum (%)
        time (float): The time period in years

    Returns:
        float: The calculated simple interest
    """
    return (principal * rate * time) / FORMULA_DIVISOR


# ============ MAIN EXECUTION ============
# Calculate simple interest using constants instead of hardcoded values
print("Simple Interest:", calculate_simple_interest(PRINCIPAL_1, RATE, TIME))
print("Simple Interest:", calculate_simple_interest(PRINCIPAL_2, RATE, TIME))
```

OUTPUT:

```
PERFORMANCE COMPARISON
================================================== ...
PS C:\Users\varun\OneDrive\Desktop\python> & "C:/Prog
e" c:/Users/varun/Downloads/Untitled-2.py
Simple Interest: 100.0
Simple Interest: 200.0
PS C:\Users\varun\OneDrive\Desktop\python>
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

Sample Code:

x = 50

y = 30

z = x + y

print(z)

Expected Output:

A Python program with clear variable names and explanatory comments

that improve readability without altering functionality.

CODE:

```
sers > varun > Downloads > untitled-2.py > ...
# ==============================================
# TASK 5: Refactoring - Enhancing Variable Nam
# ==============================================

# Define descriptive variables instead of sing
# These represent quantities that need to be s
first_number = 50       # First numeric value
second_number = 30      # Second numeric value

# Calculate the sum of both numbers
total_sum = first_number + second_number

# Display the result
print(total_sum)
```

OUTPUT:

```
PS C:\Users\varun\OneDrive\Desktop\python> & "C:/Program Files/Python312/
  - Difficult to maintain or extend

Refactored Code Benefits:
  - Descriptive variable names (first_number, second_number, total_sum)
  - Comments explain the purpose of each variable
  - Self-documenting code that's easy to understand
  - Better maintainability for future modifications

Output Verification:
  - Original: 80
  - Refactored: 80
  ✓ Output remains unchanged!
```